Java — Zero to Master

24 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction, JDK and First Program
02 Variables and Data Types
03 Operators and Type Casting
04 Control Flow
05 Methods
06 Arrays and Strings
07 Classes and Objects

Intermediate (7 topics)

08 OOP: Encapsulation, Inheritance, Polymorphism, Abstraction
09 static, final, this, super
10 Exception Handling
11 Collections Framework
12 Generics
13 File I/O and NIO
14 Packages and Access Modifiers

Advanced (6 topics)

15 Lambdas and Functional Interfaces
16 Streams API and Optional
17 Multithreading and Concurrency
18 Records, Sealed Classes, Pattern Matching
19 JDBC and Databases
20 Maven/Gradle and Testing

Master (4 topics)

21	JVM Internals and Garbage Collection
22	Design Patterns and SOLID
23	Real Projects
24	Best Practices and Interview Prep

Beginner Level

01. Introduction, JDK and First Program

Java is a statically typed, object-oriented language that compiles to bytecode and runs on the JVM ('write once, run anywhere'). Install a JDK, then compile with `javac` and run with `java`.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

02. Variables and Data Types

Primitives: `byte short int long float double char boolean`. `String` is a class. Use `var` for local type inference.

```
int age = 25;
double price = 99.5;
String name = "Arun";
var count = 10;
```

03. Operators and Type Casting

Arithmetic, relational, logical, ternary operators; widening (automatic) and narrowing (explicit cast) conversions.

```
int a = 7 / 2;          // 3
double b = 7 / 2.0;     // 3.5
int c = (int) 9.8;      // 9
```

04. Control Flow

`if/else`, `switch` (including switch expressions), `for`, enhanced `for`, `while`, `do-while`, `break`, `continue`.

```
for (int n : new int[]{1, 2, 3}) {
    System.out.println(n);
}
```

05. Methods

Reusable blocks with parameters and return type; overloading means same name with different parameters.

```
static int add(int a, int b) { return a + b; }
static double add(double a, double b) { return a + b; }
```

06. Arrays and Strings

Arrays have fixed size. `String` is immutable; use `StringBuilder` for many concatenations.

```
int[] marks = {80, 90, 70};
String s = "java";
System.out.println(s.toUpperCase() + marks.length);
```

07. Classes and Objects

A class is a blueprint; an object is an instance. Fields hold state, methods hold behaviour, constructors initialise.

```
class Student {
    String name;
    Student(String name) { this.name = name; }
    void hello() { System.out.println("Hi " + name); }
}
```

Intermediate Level

08. OOP: Encapsulation, Inheritance, Polymorphism, Abstraction

Hide data with `private` + getters/setters; reuse with `extends`; override methods for polymorphism; model contracts with `abstract` classes and `interface`s.

```
interface Shape { double area(); }
class Circle implements Shape {
    private final double r;
    Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
}
```

09. static, final, this, super

`static` belongs to the class; `final` prevents change/override; `this` is the current object; `super` refers to the parent.

```
static final double PI = 3.14159;
```

10. Exception Handling

Checked vs unchecked exceptions; `try/catch/finally`, try-with-resources, custom exceptions.

```
try (var in = new Scanner(System.in)) {
    int n = Integer.parseInt(in.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Not a number");
}
```

11. Collections Framework

`List` (ArrayList), `Set` (HashSet, TreeSet), `Map` (HashMap), `Queue` / `Deque`; choose by ordering, duplicates and lookup needs.

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Arun", 90);
marks.getOrDefault("Ravi", 0);
```

12. Generics

Type-safe containers and methods with type parameters; wildcards `? extends T`.

```
static <T> T first(List<T> list) { return list.get(0); }
```

13. File I/O and NIO

Read/write files with `Files`, `Path`, `BufferedReader`; try-with-resources closes streams.

```
List<String> lines = Files.readAllLines(Path.of("a.txt"));
```

14. Packages and Access Modifiers

Organise classes in packages; `public`, `protected`, default and `private` control visibility.

```
package com.example.app;
import java.util.List;
```

Advanced Level

15. Lambdas and Functional Interfaces

Concise anonymous functions for interfaces with one abstract method (`Runnable`, `Comparator`, `Function`, `Predicate`).

```
list.sort((a, b) -> a.length() - b.length());
```

16. Streams API and Optional

Declarative pipelines: `filter`, `map`, `collect`, `reduce`; `Optional` models absent values.

```
List<String> names = users.stream()
    .filter(u -> u.age() > 18).map(User::name).toList();
```

17. Multithreading and Concurrency

Threads, `ExecutorService`, `CompletableFuture`, locks, `synchronized`, concurrent collections and virtual threads (Java 21).

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    ex.submit(() -> System.out.println("task"));
}
```

18. Records, Sealed Classes, Pattern Matching

Modern Java: `record` for immutable data, `sealed` hierarchies, `switch` pattern matching, text blocks.

```
record Point(int x, int y) {}
String d = switch (shape) { case Circle c -> "circle"; default -> "other"; };
```

19. JDBC and Databases

Connect to databases with JDBC; always use `PreparedStatement`; connection pools (HikariCP).

```
try (var ps = con.prepareStatement("SELECT * FROM users WHERE id = ?")) {
    ps.setInt(1, id);
}
```

20. Maven/Gradle and Testing

Build tools manage dependencies and builds; test with JUnit 5 and Mockito.

```
@Test void adds() { assertEquals(5, calc.add(2, 3)); }
```

Master Level

21. JVM Internals and Garbage Collection

Class loading, heap/stack, GC algorithms (G1, ZGC), tuning and profiling with JFR/VisualVM.

```
java -Xmx512m -XX:+UseG1GC App
```

22. Design Patterns and SOLID

Singleton, Factory, Builder, Strategy, Observer; SOLID principles for maintainable code.

```
var order = Order.builder().id(1).customer("Priya").build();
```

23. Real Projects

Library management (console + JDBC), banking system, REST API with Spring Boot, multi-threaded downloader.

24. Best Practices and Interview Prep

Immutability, meaningful names, avoid null, favour composition, `equals/hashCode`, collections complexity, concurrency basics.

```
@Override public boolean equals(Object o) { /* ... */ }
```

Interview Question Bank

Q1. JDK vs JRE vs JVM?
JVM runs bytecode; JRE = JVM + libraries; JDK = JRE + development tools (javac).

Q2. `==` vs `equals()`?
`==` compares references (or primitives); `equals()` compares logical content.

Q3. Why is String immutable?
Security, caching in the string pool, thread safety and hash code stability.

Q4. ArrayList vs LinkedList?
ArrayList has fast random access; LinkedList is faster for inserts/removals at ends or via iterator.

Q5. How does HashMap work?
Hashes keys to buckets; collisions use linked lists/trees; resizes when load factor is exceeded.

Q6. Abstract class vs interface?
Abstract classes share state and code; interfaces define contracts (with default methods) and allow multiple implementation.

Q7. Checked vs unchecked exceptions?
Checked must be declared or caught; unchecked (RuntimeException) need not.

Q8. What is garbage collection?
Automatic reclamation of memory from unreachable objects.

Q9. `synchronized` vs `volatile`?
`synchronized` gives mutual exclusion and visibility; `volatile` only guarantees visibility.

Q10. What is a functional interface?
An interface with exactly one abstract method, usable with lambdas.

Capstone Project Ideas

1. Student management (console)
2. Banking application with exceptions
3. Library system with JDBC
4. Multithreaded file downloader
5. REST API with Spring Boot